Does Congress trade what it regulates?

Design report and build guide · Updated October 18, 2026

This is a running record of how the project is built and why. Each section of the build adds a chapter under "Build log". Decisions are written down at the moment they are made, including the options that were turned down, so the reasoning survives after the code changes.

1. The question

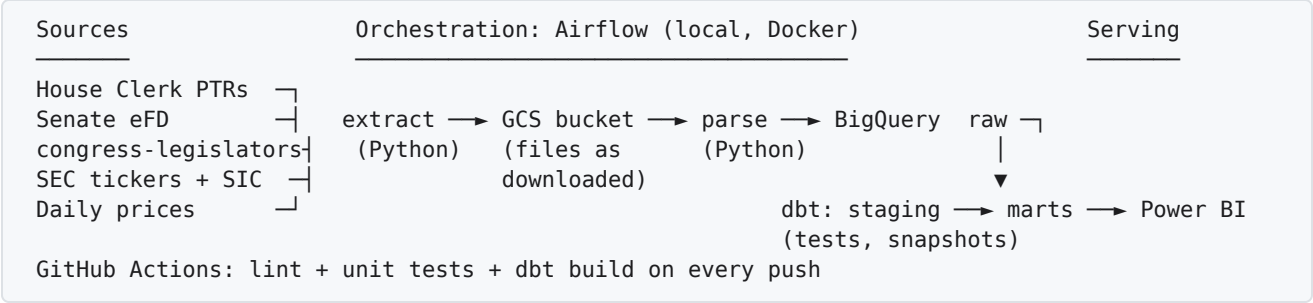
Members of Congress must disclose stock trades over $1,000 within 45 days (STOCK Act, 2012). The disclosures are public but scattered across two chamber websites, partly as scanned PDFs, and report amounts only as ranges such as "$1,001 - $15,000".

The project asks:

1. Do members trade more often in industries their own committees oversee?
2. How often are trades disclosed after the 45-day deadline, and by whom?
3. How did those trades perform against the market over 30, 90 and 180 days?
4. What does all of that look like for Michigan's delegation?

Every result is a pattern in public filings. A trade in an overseen industry is not evidence of misuse of information, and the report and dashboard will say so plainly.

2. Architecture (target)



Three storage layers, each with one job:

Layer	Where	What it holds	Why it exists
Landing	GCS bucket	Source files byte for byte (HTML, PDF, CSV, JSON)	Re-parse history without re-downloading, and prove where every row came from
Raw	BigQuery `raw`	Parsed rows, untouched values, plus load metadata	One place dbt reads from; no cleaning yet so bugs in cleaning are recoverable
Staging / marts	BigQuery, built by dbt	Cleaned, typed, tested, modeled tables	What analysis and Power BI use

3. Build log

Section 1: project skeleton and local Airflow

What was built: the folder layout, a settings module, a Docker image and docker-compose file for Airflow, a smoke-test DAG, unit tests, and a Makefile.

Verified: 4 unit tests pass, ruff is clean, the image builds, all Airflow services report healthy, and the smoke-test DAG succeeds (it read the settings and found dbt-core 1.12.6 with dbt-bigquery 1.12.1).

Decision: business logic in `src/`, DAGs stay thin

DAG files in `dags/` only say *when* and *in what order* things run. The work itself (downloading, parsing, loading) lives in the `congress_trades` package under `src/`.

- *Why:* code in `src/` can be unit-tested with plain `pytest` without starting Airflow. The Airflow scheduler also re-reads DAG files constantly, so heavy imports there slow it down. And if Airflow were ever swapped out, the package would survive.
- *Rejected:* writing logic inline in DAG files, which is common in tutorials but is hard to test and is the first thing a reviewer looks for.

The `src/` layout (instead of a top-level `congress_trades/` folder) forces tests to run against the installed package, which catches packaging mistakes early.

Decision: settings from environment variables, no extra library

`src/congress_trades/config.py` reads every machine-specific value (GCP project, bucket, BigQuery location) from environment variables into a frozen dataclass. It reports *all* missing variables at once, so a new setup fails with one clear message rather than a chain of errors.

- *Why environment variables:* the same code runs on your laptop, inside the Airflow containers and in GitHub Actions; only the variables differ. Secrets never land in the code or in git (`.env` is gitignored, `.env.example` is the committed template).

- *Why no library such as pydantic-settings:* anything installed into Airflow's environment must respect Airflow's pinned versions. A dataclass covers what we need with zero dependencies.
- *Why a User-Agent setting:* SEC EDGAR refuses requests without a contact email in the User-Agent header, and identifying ourselves to the House and Senate sites is good scraping practice.

Decision: Airflow 3.3.2 in Docker with LocalExecutor

The compose file is a trimmed copy of the official Airflow 3.3.2 one.

Kept	Removed	Why
Postgres (metadata only)	Redis, Celery workers, Flower	LocalExecutor runs tasks as scheduler subprocesses, enough for one daily DAG and much lighter on a laptop
api-server, scheduler, dag-processor	triggerer	Nothing uses deferrable operators yet; easy to add back
One-shot `airflow-init`	User-creation step	Login is disabled locally (see below)

- *Why Docker:* Airflow does not run natively on Windows and is fiddly to install on macOS. Docker gives everyone the same setup with one command.
- *Why Airflow 3:* it is the current major version, and the TaskFlow API (`@dag`, `@task` from `airflow.sdk`) makes DAGs read like normal Python.
- *Postgres here is not the warehouse.* It only stores Airflow's bookkeeping (which runs happened, which tasks failed). The project's data lives in BigQuery.
- *Login:* `SIMPLE_AUTH_MANAGER_ALL_ADMINS` makes everyone an admin with no login. That is fine on localhost and unsafe anywhere else, which is noted in the compose file.

Decision: two Python environments in one image

`docker/airflow.Dockerfile` installs:

1. our package into Airflow's environment, **using Airflow's official constraint file** so nothing we add can upgrade a library Airflow relies on;
2. dbt into its own virtualenv at `/opt/dbt-venv`.

- *Why separate dbt:* dbt and Airflow pin many of the same libraries to different versions. Installing both in one environment is the most common way local Airflow setups break. A separate venv avoids the conflict, and Airflow just calls the `dbt` command.
- *Rejected:* Astronomer Cosmos (renders each dbt model as an Airflow task). It is good, but it adds a layer to learn; calling `dbt build` from a single task is simpler to explain and enough for this project's size. It can be revisited later.

- *Editable install + volume mount:* the image installs the package in editable mode and docker-compose mounts `./src` over it, so code edits show up in Airflow immediately without rebuilding.

Decision: dbt Core 1.12 (not dbt v2)

When dbt runs it notes that "dbt v2" is now the default for new installations. The project pins **dbt Core 1.x** (`dbt-core~=1.12.0`, `dbt-bigquery~=1.12.0`) because it is the long-established open-source engine with the most documentation and community examples, and the skills transfer directly to what most employers run today. Moving to v2 later is a contained change to one line of the Dockerfile plus any model fixes.

Decision: raw files land in Google Cloud Storage

Every downloaded file is stored unchanged in a GCS bucket before any parsing, organized by source and download date.

- *Why:* House filings include scanned PDFs, and the parser will improve over time. Keeping originals means any day's data can be re-parsed with a better parser, without hitting government sites again. It is also the audit trail for the reconciliation check ("every parsed trade traces back to a file").
- *Why GCS rather than a local `data/` folder:* BigQuery loads directly from GCS for free, the files survive if the Docker setup is wiped, and the "data lake in front of a warehouse" pattern is what employers use.
- *Cost:* the whole archive is expected to be well under the 5 GB free tier.

Decision: BigQuery with billing enabled (not the sandbox)

BigQuery's no-credit-card sandbox does not allow `INSERT`, `UPDATE`, `MERGE` or `DELETE`, and its tables expire after 60 days. dbt incremental models and snapshots need `MERGE`, so the project needs a normal project with billing turned on. Actual usage should stay inside the free tier (1 TB of queries and 10 GB of storage per month); a budget alert guards against surprises.

Decision: service account key mounted read-only

Containers authenticate to Google Cloud with a service account key in `credentials/` (gitignored), mounted read-only, and found by the Google libraries through `GOOGLE_APPLICATION_CREDENTIALS`. The account gets only the roles it needs (see setup below). GitHub Actions will use a separate, keyless identity when CI is added.

How section 1 was built, step by step

This walkthrough follows the order the files were written in. Each step says what the file does, how it works, and what to look for when reading it.

Step 1: folder layout. The first decision was where things go, because every later file depends on it.

```
congress-trades/
├── dags/                   Airflow DAGs: scheduling and task order only
│   └── smoke_test.py
├── src/congress_trades/    the Python package with all real logic
│   ├── __init__.py
│   └── config.py
├── tests/                  pytest tests for src/
│   └── test_config.py
├── docker/
│   └── airflow.Dockerfile the custom Airflow image
├── credentials/            GCP key goes here (gitignored except README)
├── docs/design-report.md   this report
├── docker-compose.yaml     the local Airflow stack
├── pyproject.toml          package metadata, dev tools, lint and test config
├── Makefile                short commands for common tasks
├── .env.example            template for local settings
├── .gitignore / .dockerignore
└── README.md
```

Step 2: `pyproject.toml`. This one file replaces `setup.py`, `requirements.txt`, `pytest.ini` and a lint config.

- `[project]` names the package and requires Python 3.12 or newer, matching the Python inside the Airflow image. `dependencies` is empty for now; libraries get added only when a section needs them.
- `[project.optional-dependencies] dev` holds tools you need while developing (pytest, ruff) but the pipeline never needs at runtime, so they stay out of the Airflow image.
- `[tool.setuptools.packages.find] where = ["src"]` tells the installer the package lives under `src/`.
- `[tool.pytest.ini_options]` and `[tool.ruff]` configure the test runner and linter. Ruff's rule sets: `E`/`F` (pyflakes and style errors), `I` (import order), `UP` (modern Python syntax), `B` (common bug patterns).

Step 3: `src/congress_trades/config.py`. The settings module.

```python
@dataclass(frozen=True)
class Settings:
    gcp_project_id: str
    gcs_raw_bucket: str
    http_user_agent: str
    bq_location: str = "US"
    bq_raw_dataset: str = "raw"
```

- `frozen=True` makes settings read-only after creation, so no task can accidentally change the project id halfway through a run.
- Fields without defaults are required; fields with defaults are optional.

- `from_env(env=None)` maps each field to an environment variable name in two dictionaries (`required` and `optional`). It first collects *every* missing required variable, then raises one `ConfigError` naming all of them. Values are `.strip()`-ed so stray spaces in `.env` don't cause confusing errors.
- The `env` parameter defaults to the real `os.environ`, but tests can pass a plain dictionary instead. This is called dependency injection: the function receives its input rather than reaching out for it, which makes it easy to test.
- A last check rejects a User-Agent without an `@`, because SEC EDGAR blocks requests without a contact email and we'd rather fail at startup than in the middle of a download.

Step 4: `tests/test_config.py`. Four small tests, each proving one behaviour:

Test	What it proves
`test_reads_required_values_and_defaults`	Required values are read and defaults fill the rest
`test_optional_values_override_defaults`	Setting `BQ_LOCATION` changes the default
`test_lists_every_missing_variable`	All missing variables are named in one error; a blank value counts as missing; a present one is not listed
`test_user_agent_needs_contact_email`	A User-Agent without an email is rejected

Each test passes a dictionary into `Settings.from_env`, so none of them depend on what is set on your machine.

Step 5: `docker/airflow.Dockerfile`. Builds our Airflow image on top of the official one.

1. `FROM apache/airflow:3.3.2-python3.12` starts from the exact Airflow and Python versions we tested. Pinning the tag means the build is repeatable.
2. `CONSTRAINTS_URL` points to Airflow's published list of exact library versions known to work with 3.3.2.
3. As `root`, it creates `/opt/dbt-venv`, installs `dbt-core` and `dbt-bigquery` there, and hands ownership to the `airflow` user. `ENV PATH=...` lets tasks call `dbt` by name.
4. Back as `airflow` (never run app code as root), it copies `pyproject.toml`, `README.md` and `src/` into `/opt/project` and runs `pip install --constraint ... -e /opt/project`.

Step 6: `docker-compose.yaml`. Describes the five containers and how they start.

- `x-airflow-common: &airflow-common` is a YAML *anchor*: a block of shared settings (image, env file, environment variables, volumes, user). Each Airflow service then says `<<: *airflow-common` to copy it in, so the settings are written once.
- **Start order** is enforced with `depends_on` plus health checks: `postgres` must be healthy, then `airflow-init` must finish successfully, and only then do `airflow-apiserver`, `airflow-scheduler` and `airflow-dag-processor` start.

- `airflow-init` runs `airflow version` with `_AIRFLOW_DB_MIGRATE=true`; the image's entrypoint sees that flag and creates or upgrades Airflow's tables first, then the container exits.
- **Volumes** connect your machine to the containers: `./dags` (DAG files), `./logs` (task logs you can read locally), `./src` (live code), and `./credentials` mounted read-only (`:ro`).
- `AIRFLOW__SECTION__KEY` variables are how Airflow reads config from the environment, for example `AIRFLOW__CORE__EXECUTOR=LocalExecutor` sets `executor` in the `[core]` section.
- `AIRFLOW__CORE__EXECUTION_API_SERVER_URL` and the JWT secret are new in Airflow 3: tasks report their state to the API server over HTTP, and the shared secret signs those requests.

Step 7: `.env.example`, `.gitignore`, `.dockerignore`, `credentials/README.md`. The safety net around secrets.

- `.env.example` is committed and documents every setting; you copy it to `.env`, which is never committed.
- `.gitignore` excludes `.env`, everything in `credentials/` except its README, local data, logs and build leftovers.
- `.dockerignore` keeps the same files out of the image build, so a key can never be baked into an image by accident.

Step 8: `dags/smoke_test.py`. A throwaway DAG that proves the setup.

- `@dag(schedule=None, ...)` means it only runs when triggered by hand. `catchup=False` stops Airflow from back-filling runs since `start_date`.
- `check_settings` is a normal Python `@task`. It imports `Settings` *inside* the function: the DAG processor parses this file every few seconds, and keeping imports inside tasks keeps parsing fast.
- `check_dbt` uses `@task.bash`: the function returns a shell command and Airflow runs it. It calls `dbt --version` from the separate venv.
- `check_settings() >> check_dbt()` sets the order: dbt is only checked after settings load.

Step 9: `Makefile`. Short names for long commands: `make setup` (local venv with dev tools), `make test`, `make lint`, `make up` (refuses to start without `.env`, then builds and starts the stack), `make down`, `make logs`, `make smoke` (unpauses and triggers the smoke DAG).

Step 10: verification. What was run before handing the section over:

Check	Command	Result
Unit tests	`pytest`	4 passed
Lint	`ruff check src tests dags`	No issues
Image build	`docker compose build`	Built; dbt-core 1.12.6, dbt-bigquery 1.12.1
Stack health	`docker compose up -d`	postgres, apiserver, scheduler, dag-processor all healthy; init exited 0
DAG import	`airflow dags list-import-errors`	None
End to end	`make smoke`	Both tasks succeeded; logs show the settings and dbt versions
Web UI	`curl localhost:8080`	HTTP 200

Troubleshooting notes from testing: Docker Hub limits anonymous image pulls, so if `postgres:16` fails with "429 Too Many Requests", run `docker login` or wait and retry. Building on a network that inspects HTTPS (some corporate or campus networks) can make `pip` fail with certificate errors inside the build; on a home network this does not happen.

Section F: the public dashboard (built early, on sample data)

Johnny asked for the frontend before the pipeline sections, so it was built against generated sample data shaped exactly like the future export. When real data exists, only `frontend/data/dashboard.json` changes.

What was built: `frontend/index.html`, `styles.css`, `app.js`, a generator at `frontend/scripts/make_sample_data.py`, and the sample `frontend/data/dashboard.json` (88 fictional members, 1,479 trades).

Verified: every view loads with no console errors, no horizontal scroll at 390px phone width, and both light and dark themes render correctly (checked with headless Chromium screenshots).

Decision: a static site, not FastAPI and not only Power BI

Johnny chose this on 2026-10-18. The dashboard is plain HTML, CSS and JavaScript that reads one JSON file. A later section adds a pipeline step that exports the dbt marts to that JSON, and a GitHub Actions workflow publishes the folder to GitHub Pages.

- *Why not FastAPI:* the data changes once a day. A live API would need paid hosting and a running server, and visitors would get nothing extra for it.

- *Why not only Power BI:* most Power BI reports need a Microsoft login to view, while a recruiter can open a GitHub Pages link from anywhere. Power BI stays in the plan as the analyst-facing report.
- *Why no framework (React etc.):* the data is a few thousand rows, there is no build step to configure, and plain DOM code is easy to read in review.

Decision: compute metrics in the browser

The JSON holds row-level trades plus member and committee lookups. `app.js` computes every figure on the fly. That is what lets the year, chamber and party filters update every number instantly. At this size (about 500 KB) it is fast; if the data grew past a few MB, the export would switch to pre-aggregated tables.

Design choices

Choice	Reasoning
Lead each view with a sentence that states the finding, set in a serif	Readers see the finding first; charts are the evidence beneath it. Inspired by data-journalism layouts, not trading-app tickers
Public Sans for interface text	It is the typeface of the US federal web design system, which ties the look to government records
IBM Plex Mono for tickers, dates and amounts	Columns of figures line up, and it reads like a filing
One accent color (verdigris) reserved for "overlap"; amber reserved for late filings	Color always means the same thing. Party colors appear only as small dots so they never dominate
Amounts drawn as a ten-step band meter	Members disclose bands, not dollar amounts. Showing the band avoids implying precision that doesn't exist
Totals shown as floor–ceiling ranges	Same reason. A midpoint is used only to sort
Observed vs. expected dumbbell chart per committee	Compares each committee's overlap rate with a fair baseline, so a high number isn't mistaken for a finding when that committee simply oversees popular sectors
Copy says "patterns, not proof" in the lede and the Method page	The project's stated stance on fairness
Theme switch: Auto, Light, Dark	Auto follows the device setting. The choice is saved in the browser, and a tiny script in `<head>` applies it before the page draws, so there's no flash of the wrong theme
Sample-data banner driven by `meta.sample`	The page can never be mistaken for real findings while sample data is loaded

The trackers that inspired the layout (Capitol Trades, Quiver Quantitative, Unusual Whales) center on a feed of trades with the politician, ticker, amount and filing delay. This project keeps that feed (the Trades view) but puts the committee comparison and the late-filing

audit up front, since that's the question it exists to answer.

How the expected overlap rate is calculated

For each trade, take the member's overseen industries and add up the share of all trades (under the current filters) that fall in those industries. Averaging that over every trade gives the overlap rate you'd see if each member had traded like Congress as a whole. For one committee, the expected rate is simply the share of all trades in that committee's industries.

Run it

```
python frontend/scripts/make_sample_data.py   # regenerate sample data
python -m http.server 8000 -d frontend        # then open http://localhost:8000
```

Opening `index.html` directly from disk won't work, because browsers block `fetch()` of local files. The page says so if it happens.

4. Google Cloud setup

You need this before section 2 can talk to Google Cloud. Section 1 runs without it.

1. Create a Google Cloud project and enable billing. Set a budget alert at $5 under Billing > Budgets & alerts.
2. Create a GCS bucket in the `US` multi-region, named something like `<project-id>-congress-raw`.
3. Create a service account `congress-pipeline` and grant it:
    - BigQuery Data Editor and BigQuery Job User on the project
    - Storage Object Admin on the bucket only
4. Create a JSON key for it and save it as `credentials/gcp-service-account.json`.
5. Fill in `GCP_PROJECT_ID` and `GCS_RAW_BUCKET` in `.env`.

5. Roadmap

1. ✅ Skeleton and local Airflow
2. House disclosures: download the yearly filing index and land the raw files in GCS
3. Parse House periodic transaction reports (text PDFs first, scanned PDFs flagged)
4. Senate eFD: search, download and parse
5. Reference data: members and committees (congress-legislators), SEC tickers and SIC codes
6. The industry-to-committee mapping table (hand-built, documented)
7. Daily prices and benchmarks
8. Load parsed rows into BigQuery `raw`

9. dbt staging models and data-quality tests
10. dbt marts (star schema) and snapshots of committee membership
11. Analysis models: committee overlap, late-filing audit, returns, Michigan
12. GitHub Actions CI, plus publishing the dashboard to GitHub Pages
13. Power BI report

✅ Public dashboard frontend (built early on sample data; export step comes after section 11)